COMMON ASSIGNMENT REPORT

CSA1304 - Theory of Computation

to the award of the degree of

BACHELOR OF ENGINEERING - CSE

Submitted by

Sanjay Raghavendra (192411041)

Jayadeep J (192511304)

B Lakshman(192373010)

Under the Supervision of Guide

DR.K.Baskar

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

1. Problem Statement and Problem Formulation

1.1 Problem Statement

In airport terminals, automated ground transportation shuttles transport passengers between remote waiting bays and flight boarding gates. The shuttle operates autonomously based on discrete sensor events indicating passenger demand.

The automated shuttle operates across four functional states:

- W – Waiting Area: The shuttle is idle at the terminal bay waiting for a passenger request.
- M – Moving to Terminal: The shuttle has departed the waiting bay and is moving towards the terminal gate.
- B – Passenger Boarding: The shuttle is positioned at the boarding gate and passengers can enter or exit.
- R – Returning: Boarding is completed and the shuttle travels back to the waiting area.

The system uses two input signals:

- p: Passenger request detected.
- n: No passenger request / idle signal.

1.2 NFA Formulation

The system is represented using the formal 5-tuple NFA:

$M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{W, M, B, R\}$ $\Sigma = \{p, n\}$ $q_0 = W$

$\delta(W,p) = \{M\}$ $\delta(W,n) = \varnothing$
$\delta(M,p) = \{B\}$ $\delta(M,n) = \varnothing$
$\delta(B,p) = \{B\}$ $\delta(B,n) = \{R\}$
$\delta(R,p) = \varnothing$ $\delta(R,n) = \{W\}$

$F = Q = \{W, M, B, R\}$

A sequence is considered valid when every input produces a non-empty transition and the automaton does not enter a dead state.

2. Objective and Expected Outcomes

2.1 Objectives

- Implement the airport shuttle operation using a formal NFA model.
- Represent $\delta: Q \times \Sigma \rightarrow P(Q)$.
- Develop an interactive GUI for entering and simulating input strings.
- Provide step-by-step visualization of state transitions.
- Identify invalid transitions and dead-state conditions.
- Provide automated test cases.

- Provide mathematical explanations for transitions.
- Provide transition history, state frequency and simulation results.

2.2 Expected Outcomes

- Correctly process p and n input symbols.
- Maintain states W, M, B and R.
- Detect invalid transitions using ∅.
- Display the current state graphically.
- Provide step-by-step and automatic simulation.
- Record transition history and verify predefined test cases.
- Export simulation/audit information.

3. Requirements, Constraints and Assumptions

3.1 Functional Requirements

- FR1 – Input Acceptance: Accept p and n through buttons or typed sequences such as “p p n n” and “ppnn”.
- FR2 – Formal Notation: Enforce $\Sigma = \{p,n\}$ and $Q = \{W,M,B,R\}$.
- FR3 – Simulation: Provide RUN, STEP, UNDO, REPLAY, RESET and EXPORT.
- FR4 – Dead-State Detection: When $\delta(q,a) = \varnothing$, mark the transition INVALID and halt.

3.2 Constraints

- Python 3.10+
- CustomTkinter and Tkinter
- Python standard library
- Desktop application
- No game engine or web server dependency

3.3 Assumptions

- The shuttle initially starts at W.
- In state B, repeated passenger requests use the self-loop $\delta(B,p) = \{B\}$.

4. Application of Relevant Course Knowledge / Concepts

4.1 Finite Automata

The airport shuttle is modeled as a finite-state system with four states.

$$Q = \{W, M, B, R\}$$

4.2 NFA

$$\delta : Q \times \Sigma \rightarrow P(Q)$$

Python sets are used to represent possible destination states.

4.3 Empty Transition / Dead State

$\delta(W,n) = \varnothing$

$\delta(M,n) = \varnothing$

4.4 Self-Loop

$\delta(B,p) = \{B\}$

4.5 Extended Transition Function

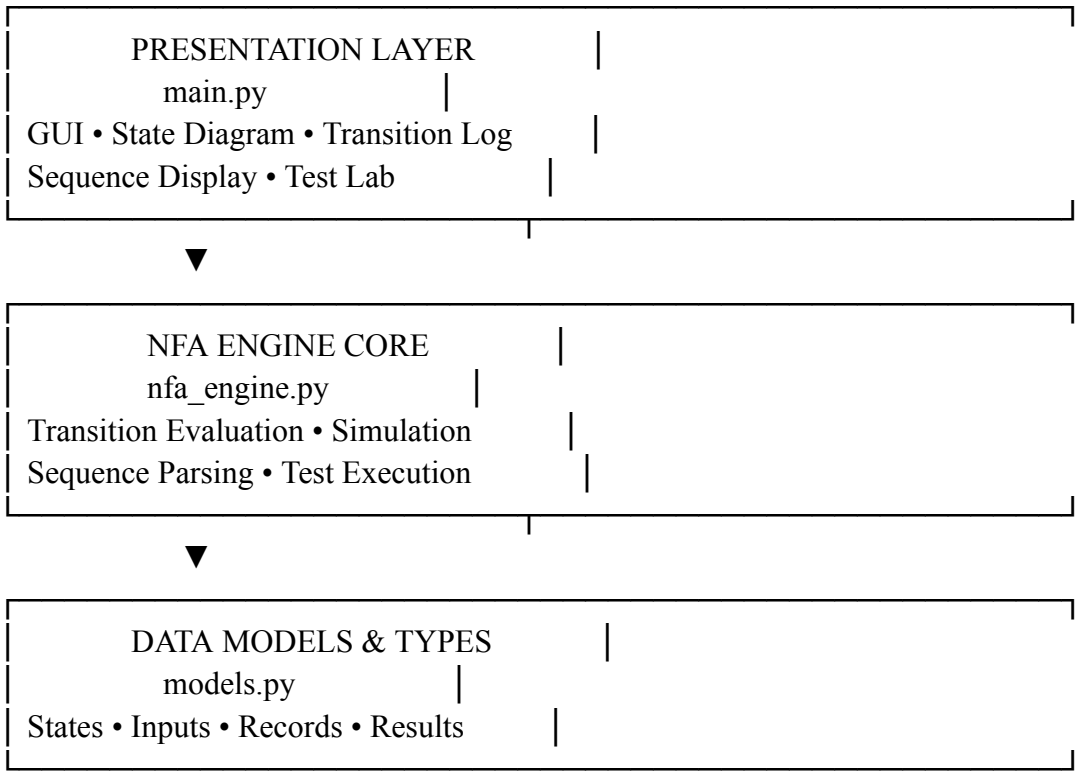
$\delta^*(q, \varepsilon) = \{q\}$

$\delta^*(q, wa) = \cup\, \delta(r,a),\ r \in \delta^*(q,w)$

The software evaluates these transitions iteratively.

5. Design / Proposed Solution / Methodology

5.1 System Architecture

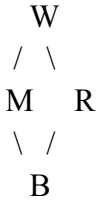


5.2 Main Components

Component	Purpose
models.py	Data models and enumerations
nfa_engine.py	Transition table, parsing, NFA simulation, dead-state detection and tests
main.py	GUI, state diagram, controls, transition log and Test Lab
launcher.py	Environment validation and application launcher

tests/test_nfa.py	Automated NFA tests

5.3 State Diagram Layout



W is the initial state, M represents forward transit, B represents boarding, and R represents the return route.

6. Algorithm / Pseudocode / Flowchart

6.1 NFA Single-Step Algorithm

ALGORITHM NFAStep(CurrentStates, InputSymbol)
INPUT: CurrentStates $\subseteq$ Q, InputSymbol $\in \Sigma$
OUTPUT: TransitionRecord
1. IF CurrentStates is empty OR Engine is halted: raise halt
2. NextStates $\leftarrow \varnothing$
3. FOR EACH state IN CurrentStates:
    Destinations $\leftarrow$ TRANSITION_TABLE[state][InputSymbol]
    NextStates $\leftarrow$ NextStates $\cup$ Destinations
4. IF NextStates = $\varnothing$: Validity $\leftarrow$ INVALID; HALT
  ELSE: Validity $\leftarrow$ VALID
5. Create TransitionRecord
6. Update history and state visit counts
7. CurrentStates $\leftarrow$ NextStates
8. RETURN TransitionRecord

6.2 Simulation Flowchart

START $\rightarrow$ Input Sequence $\rightarrow$ Initialize {W} $\rightarrow$ Read Symbol
$\rightarrow$ Compute δ(q,a) $\rightarrow$ NextStates = $\varnothing$?
YES $\rightarrow$ INVALID / HALT $\rightarrow$ END
NO $\rightarrow$ Update State $\rightarrow$ More Symbols?
YES $\rightarrow$ Read Symbol
NO $\rightarrow$ VALID $\rightarrow$ END

7. Implementation / Source Code and Environment / Tools Used

7.1 Development Environment

Component	Technology
Programming Language	Python 3.10+
GUI	CustomTkinter

Rendering	Tkinter Canvas
Testing	Python unittest
Operating System	Windows 11 / Linux compatible
Data Handling	Python standard library

7.2 Source Modules

File	Purpose
models.py	Data models and enumerations
nfa_engine.py	Transition table, parsing, simulation and dead-state detection
main.py	GUI, state diagram, controls, transition log and Test Lab
launcher.py	Environment validation and application launcher
tests/test_nfa.py	Automated NFA tests

8. Test Cases and Expected / Actual Results

ID	Test Case	Input	Expected	Actual	Status
TC-01	Standard Round Trip	p p n n	VALID	VALID	PASS
TC-02	Double Boarding Request	p p p n n	VALID	VALID	PASS
TC-03	Immediate Failure	n	INVALID	INVALID	PASS
TC-04	Mid-Route Dead Signal	p n	INVALID	INVALID	PASS
TC-05	Request After Return	p p n p	INVALID	INVALID	PASS
TC-06	Triple Boarding Loop	p p p p n n	VALID	VALID	PASS
TC-07	Double Return Signal	p p n n n	INVALID	INVALID	PASS
TC-08	Single Forward Step	p	VALID	VALID	PASS

Test Summary: 8/8 test cases passed (4 valid, 4 invalid as expected). The reported automated unit-test suite contains 43 tests.

9. Execution Screenshots / Outputs

Insert the final application screenshots in this section.

- Header: project title, simulator subtitle, status and step counter.

- Interactive Controls: p/n buttons, sequence display, RUN, STEP, REPLAY, RESET and EXPORT.
- NFA State Diagram: W, M, B, R with directed transitions, q_0 indicator and active-state highlighting.
- Transition Log: step, input, from, to, status and reason.
- Simulation Summary: input count, valid/invalid transitions, states visited, initial/final state and result.

10. Results and Validation

10.1 Functional Validation

W --p--> M
M --p--> B
B --p--> B
B --n--> R
R --n--> W

Invalid transitions are detected through the empty set $\varnothing$.

10.2 Performance

Time Complexity = $O(k \cdot |Q|)$, $|Q| = 4$

The project report records an average execution latency of less than 0.04 ms per step over 10,000 randomized symbol iterations.

10.3 State Coverage

The test suite exercises all four states W, M, B and R and the defined transitions in the transition relation.

11. Analysis, Comparison, Trade-offs and Justification

Dimension	NFA Approach	DFA Equivalent	Hardware State Machine
State Count	4 states	5 states with Trap	4 registers
Transition Definition	Uses $\varnothing$ for undefined transitions	Complete transition matrix	Hardwired logic
Self-Loop	$\delta(B,p)=\{B\}$	Explicit transition	Hold-state logic
Complexity	Compact	More explicit states	Low adaptability
Implementation	Set-based relation	Single-state transitions	Hardware dependent

Justification: The NFA model provides a compact representation of the shuttle operation. Undefined transitions can naturally be represented using $\varnothing$ without introducing an additional trap state. The mathematical engine is separated from the graphical interface, making the system easier to test and maintain.

12. Broader Considerations / SDG Relevance

SDG 9 – Industry, Innovation and Infrastructure

The project demonstrates how formal computational models can be applied to automated airport transportation systems.

SDG 11 – Sustainable Cities and Communities

Demand-driven transportation can reduce unnecessary operation and idle movement. The shuttle responds to passenger-demand signals represented by p and completion/idle signals represented by n.

13. Conclusion, Limitations and Possible Improvements

13.1 Conclusion

The Airport Shuttle NFA Interactive Simulator connects Theory of Computation concepts with a practical desktop application. The project implements $M=(Q,\Sigma,\delta,q_0,F)$, with $Q=\{W,M,B,R\}$, $\Sigma=\{p,n\}$, and $q_0=W$. The simulator provides interactive input, state visualization, step-by-step execution, transition logging, testing and validation.

13.2 Limitations

- The current system models only one shuttle.
- No ε-transitions are implemented.
- The current model uses a fixed transition relation.
- Multiple-shuttle communication is not included.

13.3 Possible Improvements

- Introduce ε-transitions to represent automatic time-based events.
- Extend the model to multiple communicating shuttles.
- Add an NFA-to-DFA subset-construction visualizer.

14. Individual Contribution of Group Members

Member	Role	Responsibilities	Contribution
[Member 1]	Lead Software Engineer & Automata Specialist	NFA formulation, NFA engine, automated testing	35%
[Member 2]	UI/UX & Graphics Designer	GUI layout, state diagram, visual feedback, color system	35%
[Member 3]	Verification & Technical Documentation	Test cases, report, viva workflow, SDG analysis	30%

15. References

1. Hopcroft, J. E., Motwani, R., & Ullman, J. E. (2006). Introduction to Automata Theory, Languages, and Computation, 3rd Edition. Pearson Addison-Wesley.

2. Sipser, M. (2012). Introduction to the Theory of Computation, 3rd Edition. Cengage Learning.

3. Schimansky, T. (2023). CustomTkinter: A modern and customizable Python UI-library based on Tkinter. GitHub Repository.

4. Python Software Foundation. (2024). Python 3.12 Standard Library Reference: Data Classes, Enumerations and Tkinter.

5. Software/Tools Used: Python, CustomTkinter, Tkinter and Python unittest.